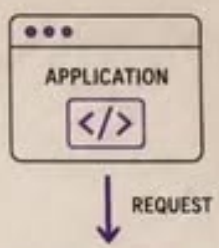
10 BACKEND CONCEPTS EXPLAINED VISUALLY

BUILD BETTER SYSTEMS.
WRITE CLEANER CODE.
THINK LIKE AN ENGINEER.

@darpan.decoded

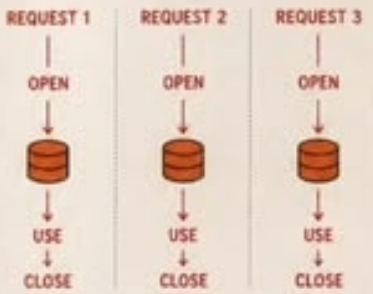
DB CONNECTION POOLING

Keep a pool of ready-to-use connections so your app stays fast & efficient.



WITHOUT POOLING

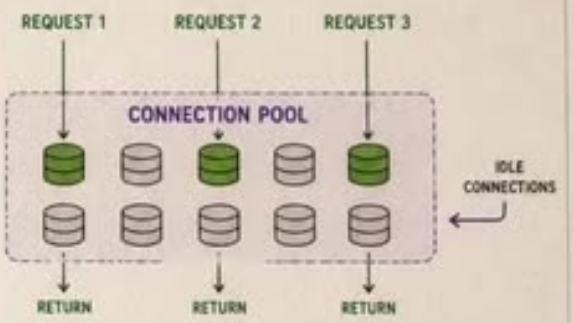
A new connection is created for every request.



SLOW & EXPENSIVE
Too much time and resources spent on opening/closing connections.

WITH CONNECTION POOLING

Connections are reused from a pool of ready-to-use connections.



FAST & EFFICIENT
Reuses existing connections.
Saves time and server resources.

HOW CONNECTION POOLING WORKS

WHY IT MATTERS

FASTER RESPONSE
No overhead of creating connection every time.

BETTER PERFORMANCE
More efficient use of database resources.

MORE STABLE
Prevents connection spikes and failures.

REUSABLE
Same connection can be used multiple times.

@darpan.decoded

BATCH PROCESSING

Batch processing executes database operations **in batches** instead of **one at a time.**

It **reduces** the number of database calls and **improves** overall performance.

HOW IT WORKS

1 COLLECT

Collect multiple requests or data into a batch.

2 GROUP

Group similar operations together.

3 PROCESS

Execute all operations in one batch.

4 COMPLETE

Return results after the batch is complete.

EXAMPLE

Instead of inserting 1000 records one by one...

Insert all 1000 records in one batch.

COMMON USE CASES

- Importing CSV files
- Bulk inserts or updates
- Processing logs & events
- Data migration & synchronization
- Generating reports & analytics

WHY IT MATTERS

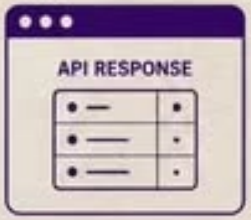
FASTER
Fewer database calls mean less overhead and lower latency.

EFFICIENT
Better resource utilization and higher throughput.

SCALABLE
Handles large volumes of data more effectively.

@ darpan.decoded

PAGINATION & SORTING



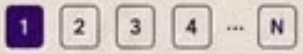
Fetching thousands of records at once **increases response time.**

It also consumes more **Heap Space** on the **Application Server.**

Pagination returns data in smaller chunks, while **sorting** organizes the results based on a **specific field.**

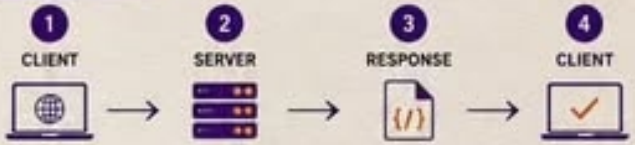
PAGINATION

Breaks large results into smaller pages.



Page Size = 10

HOW IT WORKS




```
{
  data: [..... 10 records ..... ],
  page: 2,
  size: 10,
  totalRecords: 1250,
  totalPages: 125
}
```

page → current page number
size → number of records per page
totalRecords → total number of records
totalPages → total pages available

SORTING

Arranges results based on a specific field and order.

UNSORTED (BY NAME)

ID	NAME	AGE
3	Charlie	28
1	Alice	24
2	Bob	30

SORT BY AGE (ASC)

SORTED (BY AGE ASC)

ID	NAME	AGE
1	Alice	24
3	Charlie	28
2	Bob	30

EXAMPLE API

```
GET /api/users?page=2&size=10&sort=age& order=asc
```

which page · records per page · sort by field · asc or desc

WHY IT MATTERS

FASTER RESPONSE
Smaller data chunks = faster processing and delivery

LOWER SERVER LOAD
Consumes less memory and reduces CPU utilization

BETTER USER EXPERIENCE
Organized, easy to navigate and view data in manageable pages

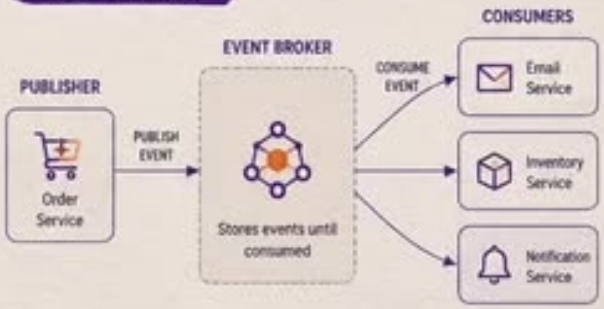
EVENT-DRIVEN ARCHITECTURE

Services communicate by **publishing** and **consuming** events instead of direct API calls.

Failure in one service doesn't impact others as events remain in the **message broker** until consumers are ready again.

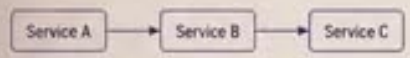
Kafka and **RabbitMQ** are common examples.

HOW IT WORKS



EVENT-DRIVEN vs TRADITIONAL (SYNCHRONOUS)

TRADITIONAL (API CALL)



- Tight coupling between services
- Failure in one service can break the flow
- Real-time calls can cause latency

EVENT-DRIVEN (EVENT BUS)

- Loose coupling between services
- Other services continue even if one fails
- Better scalability and fault tolerance

COMMON EXAMPLES

kafka

Distributed event streaming platform designed for high throughput and scalability.

RabbitMQ

Message broker that enables reliable messaging using queues and routing.

WHY IT MATTERS

LOOSE COUPLING
Services are independent and easier to scale.

RESILIENCE
Failures don't cascade; system stays stable.

SCALABILITY
Handles higher loads with ease.

FLEXIBILITY
New consumers can be added without changes.

@ darpan.decoded

API RATE LIMITING

Rate limiting controls how many requests a client can make within a specific time period.

It helps prevents abuse, protects the server from excessive traffic, and ensures fair usage for all users.

EXAMPLE

Limit: 100 requests / minute

101st request within a minute

429 Too Many Requests

HOW IT WORKS

1. **CLIENT** — Sends API request
2. **RATE LIMIT CHECK** — Server checks if request is within the allowed limit
3. **ALLOW / BLOCK** — Allow request if within limit, otherwise block it
4. **RESPONSE** — Server sends appropriate response

COMMON STRATEGIES

Fixed Window
Limits requests in a fixed time window (e.g., per minute).

Sliding Window
Considers requests in a rolling time window for smoother control.

Token Bucket
Allows bursts of requests using tokens, refills over time.

Leaky Bucket
Processes requests at a constant rate, queues the rest.

WHERE IT'S USED

- Public APIs
- Authentication Endpoints
- Payment Gateways
- Search APIs
- Email / SMS Services

WHY IT MATTERS

PREVENTS ABUSE
Stops bots and malicious users from overloading your API.

PROTECTS SERVER
Reduces server stress and prevents crashes due to high traffic.

FAIR USAGE
Ensures all users get a fair share of resources.

BETTER EXPERIENCE
Keeps the API fast, reliable and available for everyone.

@ darpan.decoded

OPTIMISTIC LOCKING

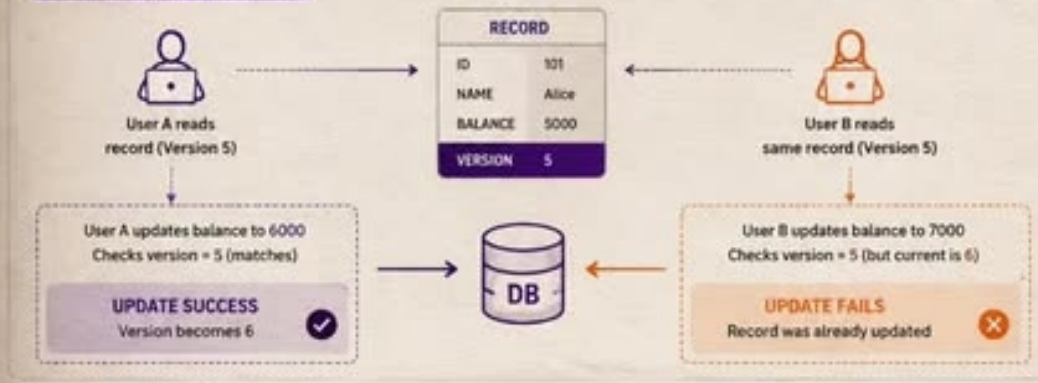
It is used to prevent **Race Conditions.**

Before updating a Record, it checks whether another txn has **modified it.**

This can be done by maintaining a **version.** If the version has changed, another txn has already updated the record, and current txn **fails.**

HOW IT WORKS

VERSION CHECK EXAMPLE



KEY BENEFITS

PREVENTS RACE CONDITIONS
Ensures two transactions don't overwrite each other's changes.

HIGH PERFORMANCE
No locks are held, improves concurrency and throughput.

REDUCES SERVER LOAD
Prevents unnecessary overwrites and rollback operations.

IDEAL FOR READ-HEAVY SYSTEMS
Best when conflicts are rare.

PESSIMISTIC LOCKING

It locks a record as soon as a txn begins, preventing other txns from modifying it until the lock is released.

This also avoids Race Conditions. But it might introduce latency if many users try to access at the same time.

WHAT IS HAPPENING?

Transaction	Lock Acquired	Record Protected	Lock Released
Txn wants to access a record	Lock is placed on the record	Other txns must wait until the lock is released	Lock is released after txn is complete

HOW IT WORKS

1 REQUEST	2 LOCK	3 BLOCK OTHERS	4 PROCESS	5 RELEASE	6 UNBLOCK
Transaction T1 requests a record.	DB places an exclusive lock on the record.	Other transactions (T2, T3, ...) must wait (are blocked).	T1 processes and updates the record.	T1 commits / rolls back and releases the lock.	Waiting transactions acquire the lock one by one.

EXAMPLE SCENARIO

User A (T1)	Lock Acquired	User B (T2)	Waiting...	T1 Complete	T2 Gets Lock
Wants to update account balance.	Record is locked by T1.	Tries to update same record.	T2 must wait until T1 releases the lock.	T1 commits and releases the lock.	T2 now gets the lock and can proceed.

ADVANTAGES

- Prevents race conditions effectively.
- Ensures data consistency and integrity.
- Simple to understand and implement.
- Best for write-heavy operations.

DISADVANTAGES

- Can cause blocking and wait times.
- Reduces concurrency and throughput.
- May lead to deadlocks in complex scenarios.
- Higher latency under heavy load.

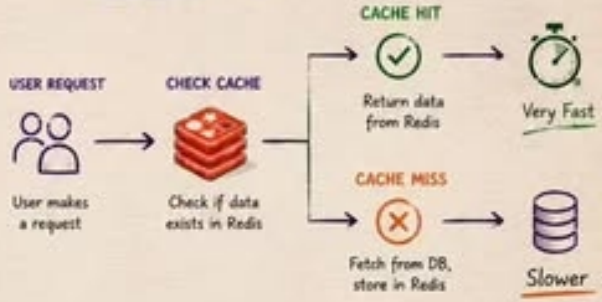
@ darpan.decoded

CACHING (REDIS)

Caching stores frequently accessed data in-memory so it can be retrieved much faster than querying the DB every time.

Redis is one of the most popular in-memory caching solution. It is fast and supports distributed caching.

HOW IT WORKS



FLOW DIAGRAM

COMMON USE CASES

KEY BENEFITS

ULTRA FAST
Data in memory = microsecond access.

SCALABLE
Handles large volume of requests with ease.

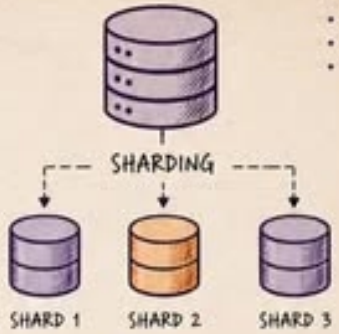
REDUCES DB LOAD
Less stress on DB, better performance and cost saving.

DISTRIBUTED
Supports distributed caching across multiple servers.

RELIABLE
Persistence options and data structures for every use case.

@ darpan.decoded

DATABASE SHARDING



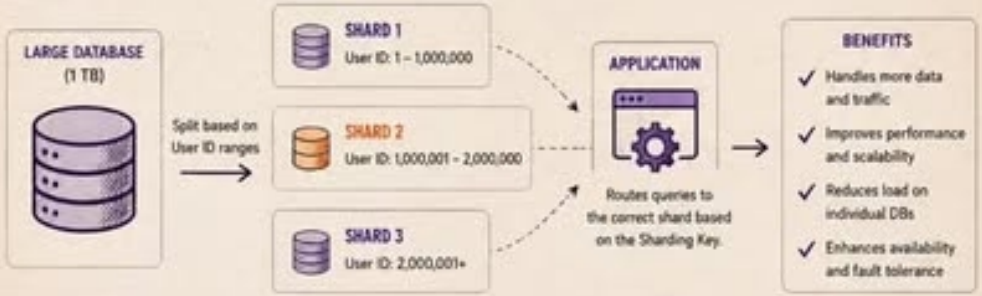
It is used for **Horizontal Scaling**.

It splits a large DB into smaller, independent DBs called **Shards**.

Each shard stores a **subset** of the data, allowing the system to handle larger datasets more efficiently.

HOW IT WORKS

EXAMPLE



KEY ADVANTAGES

HORIZONTAL SCALING
Easily scale by adding more shards as data grows.

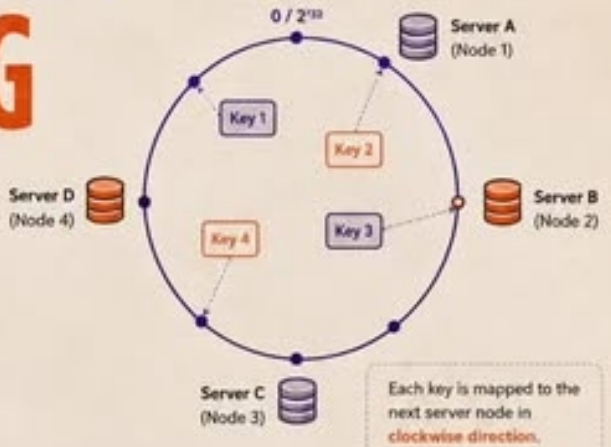
BETTER PERFORMANCE
Smaller datasets per shard mean faster queries and responses.

HIGH AVAILABILITY
If one shard fails, others remain operational, ensuring reliability.

EFFICIENT DATA MANAGEMENT
Distributes load and storage, making large-scale data management easier.

CONSISTENT HASHING

HOW IT WORKS



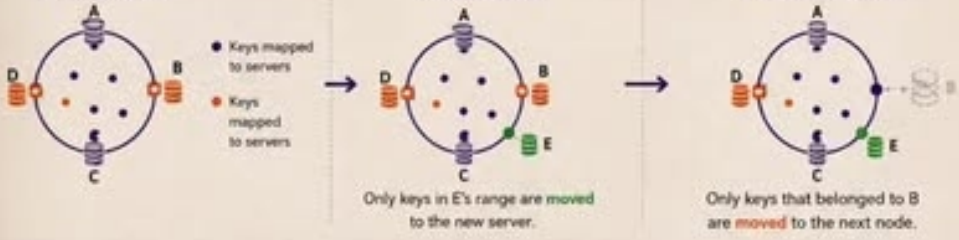
It distributes requests across multiple servers in a **circular** fashion.

When a server is added or removed, only a **small portion** of the requests are redistributed.

In traditional hashing, adding or removing a server required **a lot of redistribution**.

Each key is mapped to the next server node in **clockwise direction**.

EXAMPLE: MINIMAL REDISTRIBUTION



CONSISTENT HASHING vs TRADITIONAL HASHING

ASPECT	TRADITIONAL HASHING	CONSISTENT HASHING
ADDING A SERVER	Most / All keys are remapped	Only a small portion of keys are remapped
REMOVING A SERVER	Most / All keys are remapped	Only a small portion of keys are remapped
DISTRIBUTION	Can be uneven	More uniform distribution
SCALABILITY	Poor scalability	Excellent scalability
COMPLEXITY	Simple but inefficient	Slightly complex but efficient

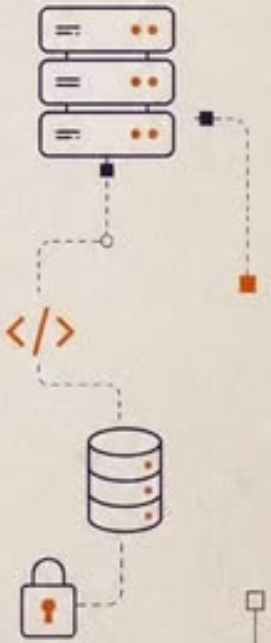
KEY TAKEAWAYS

- Minimizes data movement when servers change.
- Improves scalability and performance.
- Ideal for distributed systems and in-memory caches.
- Ensures high availability and fault tolerance.

WHERE IT IS USED

Distributed Caches | Load Balancers | CDNs | Databases | Peer-to-Peer Systems

DON'T JUST SCROLL.
SAVE THIS.



SAVE IT
So you can come back to it later.

SHARE IT
With your dev friends who'll thank you later.

APPLY IT
Build better systems. Write better code.

FOLLOW
@darpan.decoded for more such content on Backend, System Design & Tech.

ONE CONCEPT AT A TIME,
YOU BECOME 10X BETTER.

LET'S GROW TOGETHER
@darpan.decoded